# Progress Report

Thierry Sae-Lim – Norikazu Takahashi
Okayama University
June 23, 2026

**Research Objective and Goal**

The final goal of the project is to implement the distributed algorithm for PCA (Principal Component Analysis) developped by our lab and test its validity based on numerical experiment.

**Short-Term Goals**

1. Reading the paper entitled “Fast linear iterations for distributed averaging”, by Lin Xiao and Stephen Boyd.
2. Writing a Python program to simulate the algorithm (the distributed linear iteration) in the paper (Section 1).
3. Reading the paper entitled “A Novel Iterative Method for Principal Component Analysis”, by Qiuyun Cao, Kotaro Hattori, Tsuyoshi Migita and Norikazu Takahashi.
4. Implementig the Power Method from the paper in Section 2. (Optionnal: Implementing the Update Rule in Section 3)

**Progress**

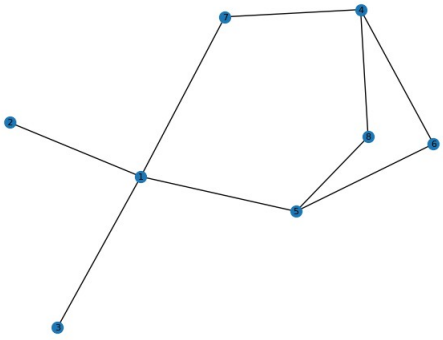
1. I read the paper, but there are some points I don’t understand: the proofs of theorem 1, the section 4.1 on the constant edge weghts, the section 6 on the computational methods, and the section 7 on the extensions.
2. I finished writing the program.
   (ア) First, I defined a simple adjacency matrix that represents an undirected graph and the initial values of each nodes x(0) to test with. I implemented a function to initalize the problem: it creates a dictionnary to store data from the adjacency matrix and the chosen initial values. In the dictionnary is stored the nodes indices (that start from 1 and not 0) as keys, and a sub-dictionnary as values. In the sub-dictionnary, given a node selected on the primary disctionnary, is stored the data of the node’s neighbors (key: “n”), the degree of the node (key: “d”), and all the taken values of the node (key: “x”).
   (イ) Next, I wrote a function that creates the weight matrix W from an instance of the problem based on the local-degrees weights (Section 4.2 from the paper).
   (ウ) And finally, I implemented the distributed linear iteration:

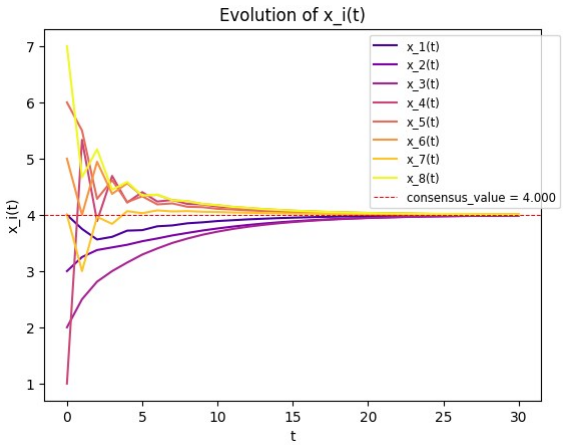
We first need to verify the convergence conditions in the paper (Section 2).

Then we can compute and add the nexts value of each node to the data.

(エ) To manipulate the data more easily, I created 2 getter functions to obtain directly the consensus value wanted, the vector x at time t.

(オ) Testing for the manually defined instance of the problem (N=8, M=9, T=30):






(カ) Everything is working on the example adjacency matrix and the initial values I manually defined previously, but I need to test if it also work on other graphs. So I implemented a function (inspired from the generation method from the paper in Section 5.1) that randomly creates a undirected graph of N nodes and M edges:

Generating the N positions of each nodes and creating the associated complete graph. Next, finding a minimum spanning tree of the graph to ensure its connexity. Then, adding edges one by one by choosing the shortest edge that isn't in the graph yet first till the graph has M edges. Converting the generated graph to its adjacency matrix to use it next. Finally, generating randomly the vector x(0) of the N initial values.

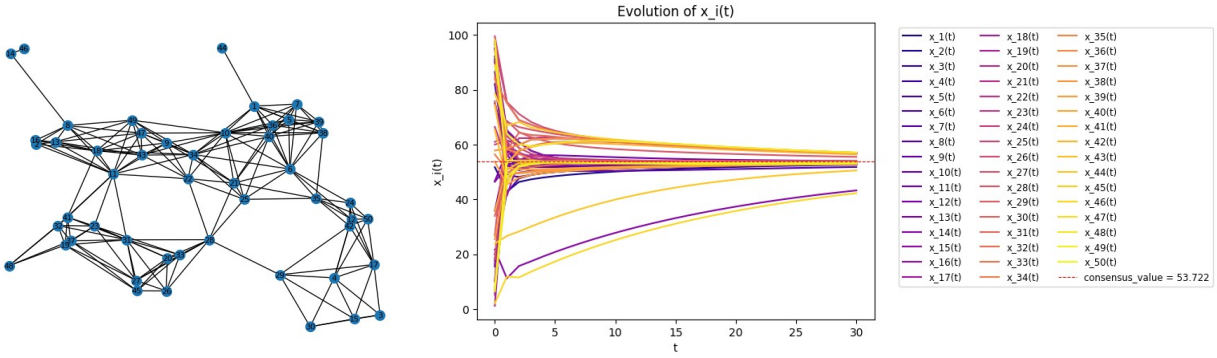
(キ) I created a function to plot the evolution of the values taken by the nodes. The plot is different depending on the parameter i:

- If i isn't set, the evolution of every agents will be shown in a single plot, it should be useful to visualize the global evolution of x over time t.
- if i is set to 0, the evolution of every agents will be shown separately.
- if i is set as a node ID, then the evolution of this agent only will be shown.

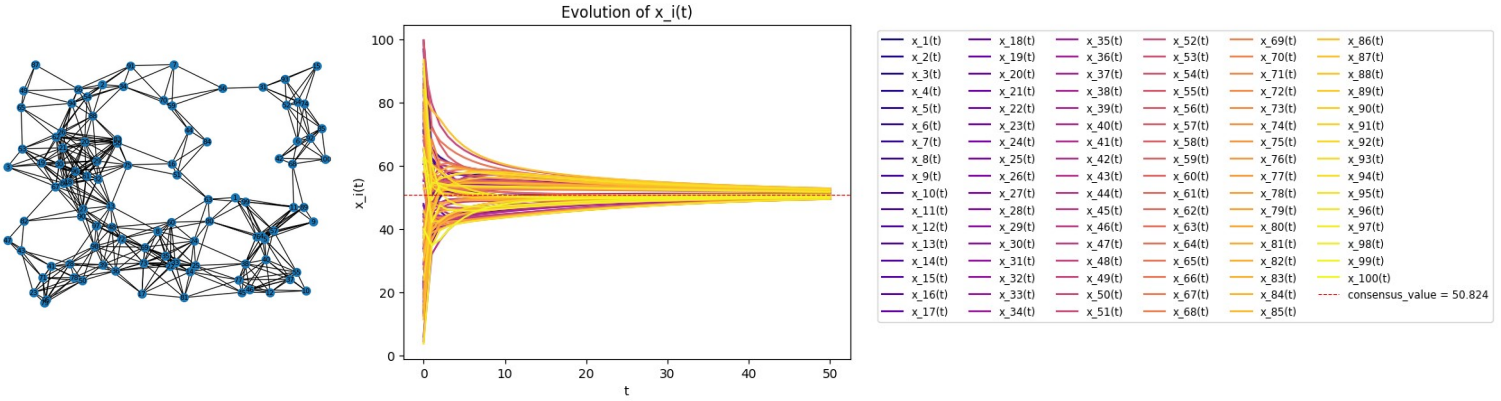
(ク) I also created a fonction to display the graph associated to an adjacency matrix, it should be useful to visualize the randomly generated graphs.

(ケ) Now I can test on randomly generated graphs, by varying the global values of N, M and the number of iteration desired T. Here are some examples:

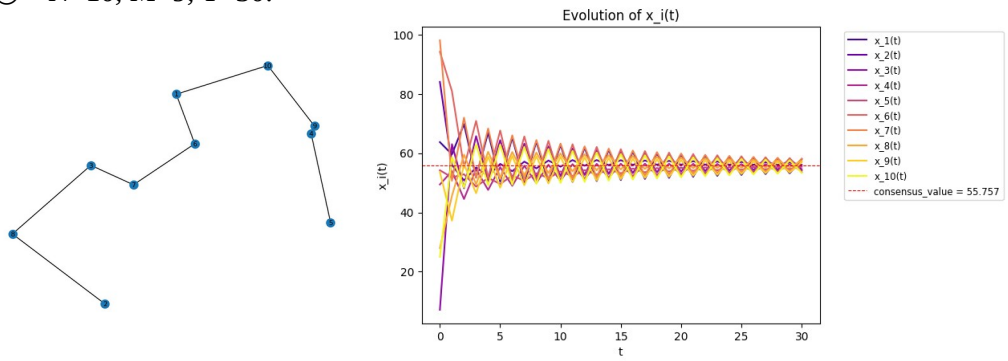
① N=50, M=200, T=30:




② N=100, M=500, T=50:




③ N=10, M=9, T=30:




(ヨ) he tests are complete and so is the short-term goal 2.

3. I read the paper, but did not read the theoretical analysis section (Section 4).
4. I finished implementig the algorithms in the paper : the classical power method, and the new update rule proposed in the paper.

(ア) First, to create an instance X of the problem, we can randomly generate NxL matrices and subtract each rows by its average value to ensure that the sum of all entries in each row of X is 0. We then need some initial normalized vectors to start with. We can simply generate random vectors then normlizing them. We next compute the covariance matrix S from X. Here's a quick reminder. We know that S is symmetric and positive semi-

definite. Hence, S can be written as S = QΛQ.T with Λ = diag(λ1 ≥ · · · ≥ λN ≥ 0). We want to find the matrix U = (u1 … uP) such as we want to minimize $\|X - U\,U.T\,X\|^2\_F$. We also know that the P firsts columns of Q, which we will call Q' = (q1 … qP), is an optimal solution of the PCA optimization problem. So the first step would be to implement functions to find the spectral decomposition of S to find the Q matrix. This can be easily done with the NumPy library.

(イ) Now that we have an optimal solution, we can implement the classical power method: the data is stored in a Python dictionnary where each key k is associated to a list of all of the estimated $k^{th}$ principal component at time t. So for example, data[3][50] is the estimation of the $3^{rd}$ principal component at the $50^{th}$ iteration. So, at each iteration for each principal component, we store the computed value. So the U matrix at the time t is the concatenation of the column vectors of each principal component at the time t: U(t) = (data[1][t] … data[P][t]). Note that each principal component is estimated after the fixed global variable T.

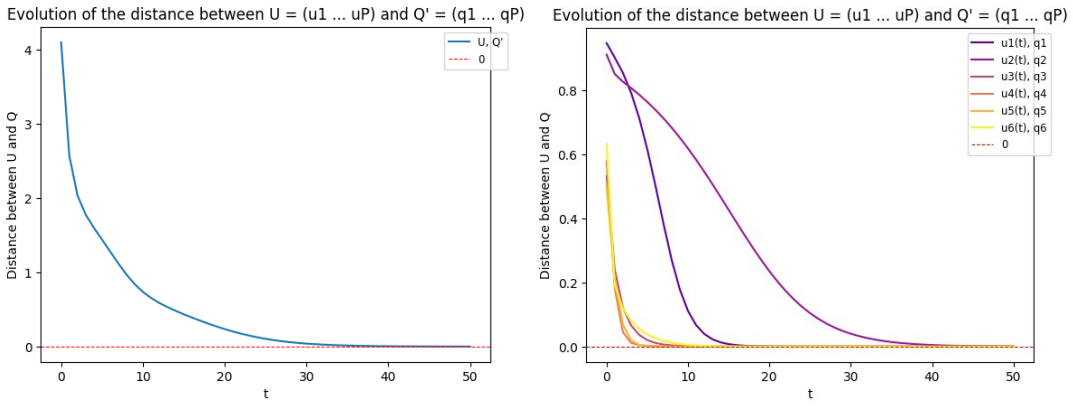
(ウ) To test the accuracy and the convergence of the power method, we can compute the distance of the vectors u1 to q1 until uP to qP. Note that a principal component is a column of the matrices, that is to say, a direction. So two columns may have the same direction but not the same orientation. In this case, vectors of U would have the same values as vectors of Q', accurate to the sign. So, the computated distances are calculated without taking the orientations of the vectors into account.

(エ) I implemented a similar plot function than in the short-term goal 2. The plot function takes an i parameter (default value is None) that determines how to plot:

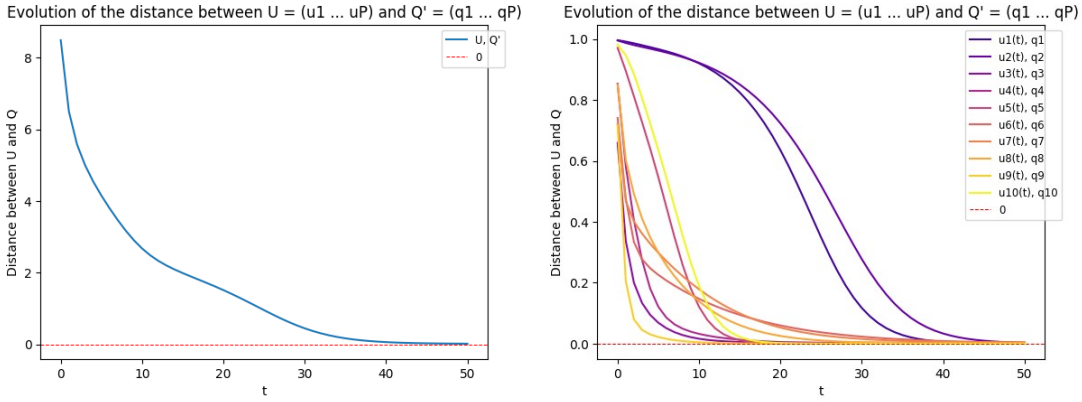
- if i=None, the function plots the total distance between U and Q'.
- if i=0, the function plots the distances between ui(T) and qi in a single plot.
- if i=-1, the function plots the distances between ui(T) and qi separately.
- if i is between 1 and P, the function plots the $i^{th}$ principal component only.

Here are some example with the evolution of the distances between U and Q' and each estimated principal component and its optimal value:

① N=10, L=15, P=6, T=50:



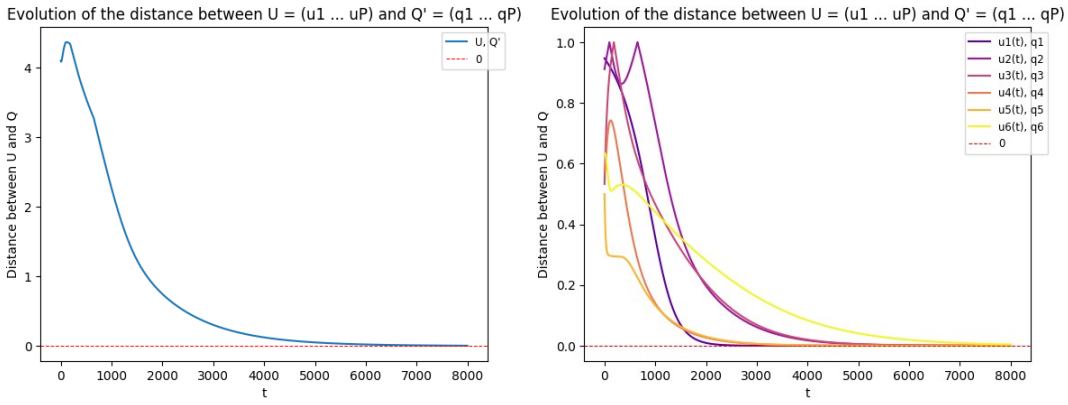
② N= 20, L= 35, P=10, T= 50:



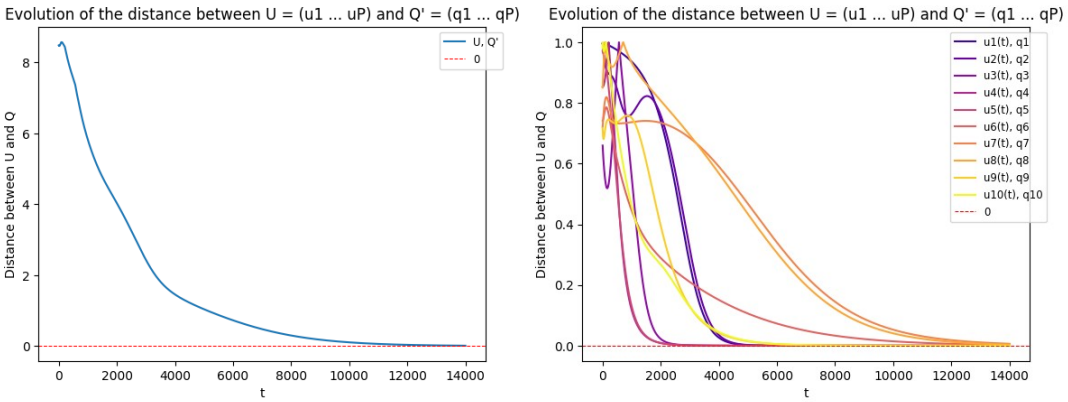

(オ) We can see that the implemented power method is converging to the optimal solution. Now we can implement the update rule proposed in the paper. Note that in the function's loop, we normalize ui(t+1) after computing it, even if the direction is converging to the optimal direction, because we're computing the distance between vectors to verify the correctness of the solution.

(カ) Here are some plots with the same instances X and initial vectors with fixed update rule parameters, k1=0.2, k2=0.4, epsilon=0.1:

① N=10, L=15, P=6, T=8000:




② N=20, L=35, P=10, T=14000:



(キ) Although convergence speed isn’t the goal of the algorithm, we can observe that the update rule has a much slower convergence speed than the classical power method. A more in-depth study on the update rule parameters’ choice (k1, k2 and epsilon) can be conducted. Short-Term goal 4 finished.